

Passation développeur — points à finir (TODO(dev))

Ce dépôt est un **squelette / gros plan**. L'architecture, les interfaces et la logique métier déterministe (classification des forfaits, règles d'anomalies, géométrie des zones) sont en place et **testées**. Restent à raccorder les briques qui dépendent de l'environnement réel (modèles, caméras, caisse, UI).

Lister tous les points : `grep -rn "TODO(dev)"` .

Priorité 1 — chaîne de perception (ai/)

- [] Fournir les poids YOLO (ai/models/) : détecteur véhicules + détecteur plaques.
- [] pipeline/detector.py — brancher l'inférence ultralytics.
- [] pipeline/tracker.py — brancher ByteTrack/DeepSORT (track_id stable).
- [] pipeline/camera.py — ouverture/lecture RTSP + reconnexion.
- [] pipeline/lpr.py — OCR (EasyOCR/PaddleOCR) + affinage plaques marocaines.
- [] run.py — boucles de frames (un thread par caméra) + arrêt propre.
- [] config.yaml — **calibrer** lignes d'entrée/sortie et polygones de zones.
- [] pipeline/events.py — file de retry locale si le réseau tombe.

Source du « forfait payé » — caisse intégrée (pas de POS)

La station n'a **pas de système de caisse**. On utilise donc une ****caisse intégrée**** : l'opérateur crée un ticket à l'encaissement (page « Caisse » du dashboard → POST /tickets). Ce ticket est la source de vérité du forfait payé, rapprochée automatiquement de la transaction détectée (services/reconciliation.py, implémenté + testé). Le jour où un vrai POS arrive, il suffira d'alimenter la table tickets via un connecteur — le reste ne change pas.

Priorité 2 — logique serveur (backend/)

- [] services/ingestion.py — résolution employé via badge NFC (_on_badge).
- [] services/ingestion.py — idempotence des événements (rejeux réseau).
- [] services/rapport.py — reste à ajouter le **graphique horaire** dans le PDF et les **comparaisons** (vs veille / semaine précédente / moyenne mensuelle). (Agrégations + tableau PDF déjà implémentés et testés.)
- [] services/notification.py — appel réel WhatsApp Business API.
- [] Migrations **Alembic** (remplacer create_all de db/seed.py).
- [] Durcir l'auth (rôles, expiration, rate-limiting) avant production.
- [] Monter le stockage médias en statique (app.mount("/media", ...)).

Priorité 3 — dashboard (frontend/)

- [] Câbler chaque page aux endpoints (URLs prêtes dans `src/api/client.js`).
- [] WebSocket temps réel dans `Dashboard/LiveView`.
- [] Flux vidéo (HLS/WebRTC) dans `LiveView`.
- [] Graphiques Recharts (volume horaire, répartition forfaits, perf employés).
- [] PWA : service worker + notifications push (manifest déjà présent).

Déjà fait et testé

- Modèle de données complet (SQLAlchemy + schéma SQL de référence).
- **Classification du forfait effectué** (zones + durée) — `services/classification.py`.
- **Règles de détection d'anomalies** (6 cas du cahier des charges) — `services/anomalie.py`.
- **Rapprochement caisse ↔ transaction** (plaque puis proximité temporelle) — `services/reconciliation.py`.
- **Clôture de transaction** : classification + rapprochement + persistance des anomalies — `services/ingestion.py`.
- **Caisse intégrée** : modèle tickets, API POST/GET/annuler, page Caisse.
- **Rapprochement manuel** : POST `/tickets/{id}/rapprocher`, page Rapprochement
(associer un ticket ouvert à une transaction non appariée + recalcul anomalies).
- **Paramètres configurables** (parametres) : fenêtre de rapprochement réglable depuis l'écran Configuration.
- **Gestion des services/forfaits** : créer (POST), modifier (PUT), supprimer (DELETE, refusé si référencé) depuis l'écran Configuration. La classification et les anomalies sont **pilotées par la base** (ForfaitDef) → tout nouveau service fonctionne de bout en bout, sans toucher au code.
- **Multi-emplacements** : modèles sites + bays, transactions rattachées à une baie. Endpoints `/sites`, `/bays` (état, lavage en cours, file, temps moyen, staff), `/dashboard/aperçu` (4 cartes + delta vs veille), `/dashboard/wash-details`. Dashboard refondu (style « Clean It ») avec sélecteur de site, Wash Details, Wash Bay Stations (onglets), Package Analytics, Recent Events. Pages Bay Management & Customer Management.
- **Édition des baies/sites** : POST `/sites`, POST `/bays`, PATCH `/bays/{id}` (hors service / staff). Bay Management éditible (bascule état, +/- staff, ajout).
- **Queue Management + mode manuel** (sans caméras) : GET `/queue`, POST `/queue/demarrer` (ticket → baie), POST `/queue/terminer`. La station peut

fonctionner entièrement à la main en attendant l'installation des caméras.

- **Paielements** : GET /payments (Paid/Pending, totaux, par site). Page Payments + panneau « Transactions » sur le dashboard.
- **Temps réel (WebSocket)** : /ws/live authentifié ; le backend diffuse un signal {"type": "update"} aux points de mutation (événements IA, file d'attente, tickets). Côté frontend, LiveProvider déclenche le re-fetch automatique (dashboard, stations, file, transactions). Validé de bout en bout par test E2E.
- **Rôles & accès** : rôles admin / manager / caissier + site_id sur l'utilisateur. require_role(...), GET /auth/me, gestion des utilisateurs admin (/users). Frontend : menu adapté au rôle, page Utilisateurs (admin), garde de route.
- **Sécurité multi-site (filtrage imposé)** : dépendance resolve_site — un manager/caissier rattaché à un site est FORCÉ sur son site, quel que soit le site_id demandé (vérifié : un manager demandant un autre site reçoit quand même le sien). Appliquée à dashboard, bays, payments, queue, employés, présence ; la liste des sites est aussi filtrée. TODO(dev) : étendre aux écritures (créer/éditer sur un autre site).
- **Inventaire** : produits (stock par site + seuil d'alerte). Endpoints /produits (GET filtrable + sous_seuil, POST, PATCH, mouvement +/-, suppression logique). Page Inventory (ajustement stock, alerte réappro, par site).
- **Consommation par forfait** : forfait_produits (recette : 1 unité tous les N lavages). À chaque lavage clôturé, le stock du produit (au site du lavage) est décrémenté automatiquement (_consommer_produits). Endpoints GET/PUT /forfaits/{id}/consommation ; éditeur de recette dans Configuration. Vérifié : 8 lavages Premium → 2 unités de cire consommées, isolation par site. La recette peut aussi être saisie directement dans le formulaire « Ajouter un service » (produits + quantité en même temps que le forfait).
- **Emplacement des anomalies** : chaque anomalie renvoie son site (via la transaction→baie→site, ou l'employé pour les alertes RH) + la plaque ; affiché dans la page Anomalies et Recent Events. Filtré par site pour les utilisateurs rattachés (multi-site).
- **Page Employés + classement** : GET /employees/performance (véhicules, temps moyen, conformité, revenus, score, trié) + écran Employés.
- **Employés rattachés à un site** : site_id sur l'employé ; liste, classement

et effectif filtrables par site ; affectation modifiable depuis l'écran Employés

(POST /employes, PATCH /employes/{id}).

- **Identification employé** : par **badge NFC** (badge_nfc_id) OU par **couleur de gilet** (couleur_gilet) — _on_badge résout l'employé selon le champ fourni.
Vision : ai/pipeline/vest.py (couleur dominante du torse + matching de couleur, testé) émet EventIn.couleur_gilet.
- **Pointage (selfie horodaté)** : POST /pointage (multipart selfie) enregistre l'HEURE SERVEUR (autoritaire, anti-antidatage) + filigrane horaire sur l'image (Pillow). Page Pointage : capture caméra en direct (getUserMedia) + historique. Médias servis via /media (StaticFiles).
- **Horaires** : ouverture des sites (GET/PUT /sites/{id}/horaires) et créneaux de travail des employés (GET/PUT /employes/{id}/horaires), éditables via un éditeur hebdomadaire (Config = ouverture ; Employés = travail).
- **Présence / ponctualité / productivité** : GET /employes/presence croise planning (créneaux) + pointages (arrivée/départ) + lavages effectués → statut (présent/absent/non planifié), retard, temps présent, temps actif, temps mort, productivité. Page « Présence » (date + site).
- **Anomalie « lavage hors horaires »** : à la finalisation, l'heure d'entrée est comparée aux horaires d'ouverture du site (via la baie) ; si hors ouverture (ou jour fermé) → anomalie MOYENNE.
- **Nettoyage des transactions orphelines** : job horaire (services/maintenance.py) qui clôture en « abandonnée » les lavages « en_cours » plus vieux que delai_abandon_heures (défaut 4 h) — évite de gonfler le compteur des en-cours.
- **Verrou anti-concurrence** : _rapprocher_ticket verrouille le ticket choisi (with_for_update, effectif en PostgreSQL) et revérifie son statut avant de l'associer, pour éviter un double rapprochement en cas de clôtures simultanées.
- **Alertes RH retard/absence** : services/alertes.py. Retard détecté au pointage d'arrivée (au-delà de la tolérance configurable) ; absences détectées par un scan planifié (toutes les 15 min) des employés planifiés non pointés. Chaque alerte = anomalie (liée à l'employé) + diffusion temps réel + log (WhatsApp à câbler). Seuils configurables (tolerance_retard_minutes, delai_absence_minutes).
- **IA vidéo (finition)** : Tracker (ByteTrack via ultralytics) + detect_video.py

avec suivi → comptage fiable (anti-doublon) et émission optionnelle d'événements

ENTREE vers le backend (--emit). Testable sur une vidéo sans caméras.

- **Mode test vidéo (IA)** : Detector (ultralytics) et CameraStream (RTSP ou fichier) fonctionnels + ai/detect_video.py — teste la détection de véhicules sur une vidéo avec YOLO pré-entraîné, sans caméras installées.
- **Rapport journalier** : agrégations SQL (KPI, répartition, perf employés) + génération PDF ReportLab (résumé, forfaits, tableau employés) — testé.
- **KPI dashboard réels** (/dashboard/kpi, /dashboard/en-cours).
- **Développement sans caméras** : simulateur d'événements (ai/simulator.py) + jeu de démo multi-jours (app/db/demo.py) — validés de bout en bout.
- **Pages câblées** : Dashboard (KPI + en cours), Historique, Anomalies, Caisse, Rapprochement, Configuration (forfaits + fenêtre).
- **Géométrie zones/lignes** (franchissement, point-dans-polygone) — ai/pipeline/zones.py.
- Contrat d'événements IA ↔ backend.
- Squelette d'API (auth JWT, events, dashboard, transactions, anomalies, employés, forfaits, rapports, WebSocket).
- Structure frontend (routing, auth, layout, pages).
- Tests unitaires : pytest backend/tests et pytest ai/tests.

Ordre de mise en route conseillé (phases du cahier des charges)

1 Infra caméras/réseau/edge → 2. Détection+comptage → 3. LPR → 4. Classification
+ anomalies → 5. Suivi employés (NFC) → 6. Dashboard + rapports → 7. Tests terrain
→ 8. Production.